

Team 03 · Accessibility Agent: Progress Report

Date: 2026-05-17 **Author:** Ramon (with Claude Code)

Starting Point

Ramy built the full LangGraph agent pipeline (12 node files, graph wiring, session management) but it had never completed a clean end-to-end run. The code had three categories of issues: missing dependencies, critical logic bugs, and multiple crash points across the graph.

Phase 1: Unblock & Fix Core Bugs

1.1 `call_llm_simple` always falling back to defaults (CRITICAL)

Problem: Both pre-agents (Profile Agent, Space Type Agent) called `call_llm_simple()` which routed through `_normalize_llm_decision()`. That function expected `{"action": "final", "tool_calls": [...]}` but pre-agents return free-form JSON like `{"profile_type": "wheelchair_user"}`. The normalization always threw `RuntimeError` · silently caught · both agents fell back to hardcoded defaults every time. The LLM call was wasted.

Fix: Anthropic path in `call_llm_simple` now calls the API directly and parses free-form JSON, bypassing `_normalize_llm_decision`.

1.2 `_call_anthropic` message format mismatch

Problem: `_call_anthropic()` called `msg.get("role")` which fails on LangChain `HumanMessage` objects from the `add_messages` reducer.

Fix: Handle both dicts and message objects, map `"human" → "user", "ai" → "assistant"`.

1.3 `_normalize_llm_decision` `KeyError` on `tool_name` vs `name`

Problem: LLM returned `"tool_name"` instead of `"name"` in tool call dicts.

Fix: Added `_extract_tool_name()` helper checking `name`, `tool_name`, and `function` keys.

1.4 Pre-agents failing on nested LLM responses

Problem: LLM sometimes wraps responses in extra layers like `{"accessibility_analysis": {"profile": {...}}}`.

Fix: Both agents now search up to 2 levels deep for the expected keys.

Phase 2: Crash Prevention (try/except wrappers)

Added graceful error handling across all nodes so failures return valid update dicts instead of killing the graph:

```
| Node | Change | --- | main.py | try/finally for MCP cleanup || reason.py | try/except around  
call_llm() || tools.py | try/except MCP calls, guard empty pending_tool_calls, log JSON parse  
failures || add_objects.py | try/except MCP calls || collision.py | try/except around json.loads and  
check_collision || visibility.py | try/except around analysis || path_analysis.py | try/except  
around analysis || reachability.py | try/except around analysis || orientation.py | try/except around
```

Phase 3: State & Routing Bugs

3.1 · _keep_last reducer bug: object_to_place never clears

Problem: _keep_last(old, None) = old · setting state fields to None doesn't clear them. reason.py set object_to_place = None to indicate "done", but the old value persisted · duplicate placements.

Fix: Use {} for dicts and [] for lists instead of None in reason.py and add_objects.py.

3.2 · Infinite loop in analysis-only runs

Problem: When LLM says "action": "final" (no objects placed), collision violations still triggered "adjust" routing · back to reason · LLM says "final" again · infinite loop.

Fix: _route_after_group1 and _route_after_group2 only return "adjust" if objects were actually placed.

3.3 · Infinite adjustment loop after placement

Problem: Structural violations (e.g., bathroom turning radius) persisted forever, looping collision · adjust · reason indefinitely.

Fix: Added adjustment_count to state with MAX_ADJUSTMENTS = 3. After 3 attempts, continues to scoring regardless.

3.4 · layout_json_string not propagating through parallel nodes

Problem: After add_objects updated furniture, the value was lost by parallel fan-in.

Fix: Changed layout_json_string from str to Annotated[str, _keep_last].

3.5 · _slim_layout stripping fields needed by GH

Problem: layout_json_string used a slim version that stripped out line, structure, mep. GH tools need them.

Fix: layout_json_string stores the full layout; slim version used only in LLM prompt.

3.6 · Scoring weights not normalized

Problem: space_config could inject tool_weights that don't sum to 1.0.

Fix: Normalize weights after merging.

Phase 4: Score Not Updating After Placement

Problem: Score stayed at 60.2 despite moving 12 objects. The analysis nodes re-ran but collision score was always 0.

Root cause: Python collision node stored results under grid_meta key, but scoring.py looked for grid. Binary scoring: any hard violation · score = 0.

Fix:

- `scoring.py` checks both `grid_meta` and `grid`
 - Changed from binary to proportional scoring: $\text{score} = 100 * (1 - \text{blocked_pct} * 3 - \text{warning_pct} * 1)$
-

Phase 5: Doors Lost in Output

Problem: After furniture placement, MCP `place_objects` returned a full layout missing doors ($3 \cdot 0$).

Fix:

- `add_objects.py` and `tools.py`: merge missing layers (doors, windows, mep, structure, outline) from current state when MCP returns a full layout
 - Checkpoint: structural integrity check compares with `original_layout`, auto-restores lost layers
 - Door change detection alerts the user of removed/modified/added doors
-

Phase 6: Collision Score Dominated by Walls

Problem: Walls generated huge clearance violation zones along the entire perimeter, making scores very low even with good furniture placement. The agent can't move walls.

Fix: `scoring.py` now separates violations by `object_type`:

- **Structure (walls):** penalized at 20% weight (not actionable)
 - **Furniture/MEP:** full penalty (actionable by the agent)
-

Phase 7: Viewport Toggle System

7.1 · Created `set_viewport` GHPython component

File: `gh/set_viewport.py`

Lightweight layout renderer · receives layout JSON and a display mode, outputs Rhino geometry for real-time viewport preview. Modes: `all`, `rooms`, `furniture`, `doors`, `structure`, `outline_only`, `none`.

7.2 · Interactive checkpoint toggle

Added to `user_checkpoint_node` in `graph.py`:

```
Viewport:
1 = BEFORE layout (original)
2 = AFTER layout (disabled if no changes)
3 = + Collision overlay
4 = + Visibility overlay
5 = + Path overlay
0 = Clear overlays (layout only)
```

- 1/2 switch the active layout (before/after)
- 3/4/5 send the active layout to `collision-detector-grid` as base + call the analysis tool on top

- Tracks which layout is "active" · overlays apply to it

7.3 ·set_viewport "pending" fix

Problem: MCP call to `set_viewport` hung forever, blocking the pipeline.

Fixes:

- info output changed from plain string to valid JSON (Swiftlet requirement)
 - Added optional `timeout` parameter to `mcp_client.call_tool()`
 - `set_viewport` calls use 10s timeout
 - Auto-disabled for session after first failure, falls back to `collision-detector-grid`
 - Checkpoint auto-send wrapped in try/except
-

Phase 8: Score Comparison & ANSI Colors

8.1 · Previous vs current score display

Added `previous_scoring` to `AgentState`. Each checkpoint visit snapshots the current scoring. Next visit shows deltas:

```
LAYOUT SCORE: 72.3/100  Grade: C  · +12.1
Previous:      60.2/100
Score breakdown:
  collision      85.0/100  (weight 0.30, +25.50)  · +25.0
  visibility     68.2/100  (weight 0.25, +17.05)  · -2.3
  path          70.0/100  (weight 0.20, +14.00)  = no change
```

8.2 · ANSI color coding

- **Green** (·): score improved
 - **Red** (·): score worsened
 - **Dim** (=): no change
 - Score values colored: green ·80, yellow 50-79, red <50
 - Collision violations in red, furniture changes in cyan/yellow, integrity warnings in yellow
-

Phase 9: Smart Suggestions

Auto-generated action suggestions based on the lowest-scoring tools:

```
Suggestions:
  s1 = Fix collisions (cnc_machine, workbench_01...)
  s3 = Improve path accessibility
```

- Only appear for tools scoring < 80
 - Collision suggestions name the specific furniture causing violations
 - Selecting `s1` sends the prompt directly to the reason node as a user instruction
 - Generic fallback if score < 80 but no specific tool data available
-

Files Modified

| File | Changes | |---|---| | `_runtime/llm.py` | `call_llm_simple` Anthropic path, message format handling, `.replace()` instead of `.format()` | | `_runtime/mcp_client.py` | Optional `timeout` parameter on `call_tool()` and `_rpc()` | | `graph.py` | `AgentState` (new fields: `original_layout`, `placement_history`, `previous_scoring`), `build_user_checkpoint_node` (viewport toggles, score comparison, ANSI colors, smart suggestions, integrity check, MCP timeout), routing fixes | | `nodes/reason.py` | `_keep_last` fix (`{}` instead of `None`), `try/except` | | `nodes/tools.py` | Layer preservation, guard empty tool calls, `try/except`, logging | | `nodes/add_objects.py` | Layer preservation, placement history tracking, no-op move filter, `_keep_last` fix | | `nodes/collision.py` | `try/except` wrappers | | `nodes/visibility.py` | `try/except` wrappers | | `nodes/path_analysis.py` | `try/except` wrappers | | `nodes/reachability.py` | `try/except` wrappers | | `nodes/orientation.py` | `try/except` wrappers | | `nodes/scoring.py` | Proportional scoring, `grid_meta/grid` lookup, structure vs furniture separation, weight normalization | | `nodes/profile_agent.py` | Nested response handling, fallback logging | | `nodes/space_type_agent.py` | Nested response handling, fallback logging | | `main.py` | `try/finally` MCP cleanup |

Files Created

| File | Purpose | |---|---| | `gh/set_viewport.py` | GHPython component for viewport layout rendering (MCP tool) |

Open Issues

- 1 **set_viewport as MCP tool** · May still stay "pending" if the Swiftlet Result cluster isn't wired correctly in GH. Workaround: 10s timeout + fallback to `collision-detector-grid`.
- 2 **Viewport overlay visibility** · `set_viewport` and analysis tools are separate GH components. If `set_viewport` doesn't respond, only the analysis overlay shows. Current approach: use `collision-detector-grid` as layout base for overlay views.
- 3 **place_objects format mismatch** · LLM sometimes sends malformed input for the compact string format. Warning logged but silent failure possible.